

Lab04 Solution

30/01/2026

Exercise

Write C program to create an array of size n using dynamic memory allocation to store n integers and then sort the elements of an array using **Bidirectional Selection Sort**. In this method, in each pass find the **minimum** and **maximum** elements from the unsorted array. Place the minimum at the left and the maximum at the right. Moreover, print the array before and after sorting [CO-1].

Pseudo code for Bidirectional Selection Sort

BIDIRECTIONAL-SELECTION-SORT(A, n)

```
1  left = 1
2  right = n
3  while left < right
4      minIndex = left
5      maxIndex = left
6      for i = left to right
7          if A[i] < A[minIndex]
8              minIndex = i
9          if A[i] > A[maxIndex]
10             maxIndex = i
11         // Swap minimum to left position
12         temp = A[left]
13         A[left] = A[minIndex]
14         A[minIndex] = temp
15         if maxIndex == left // Max was at left, now moved to minIndex
16             maxIndex = minIndex
17         // Swap maximum to right position
18         temp = A[right]
19         A[right] = A[maxIndex]
20         A[maxIndex] = temp
21         left = left + 1
22         right = right - 1
```

Demonstration of pseudo code with sample input

Input

Enter the size of array: 6

Enter elements of array: 64 25 12 22 11 33

Execution step according to pseudo code:

Initial Array: 64 25 12 22 11 33

Pass 1:

initially left = 0 and right = 5 // since n is 6 here

Find min = 11 at index 4 then minIndex = 4, find max = 64 at index 0 then maxIndex = 0

Swap 11 with element at index left. i.e., index 0

Array become: **11** 25 12 22 **64** 33

Now max 64 moved to minIndex after swap. So, assign minIndex to maxIndex

Swap 64 with element at index right. i.e., index 5

Array after Pass 1: **11** 25 12 22 33 **64**

Pass 2:

left =1 and right = 4

Find min = 12 at index 2 then minIndex = 2, max = 33 at index 4 then maxIndex = 4

Swap 12 with element at index left. i.e., index 1

Swap 33 with element at index right. i.e., index 4 (already at index 4)

Array after Pass 2: 11 **12** 25 22 **33** 64

Pass 3:

left =2 and right = 3

Find min = 22 at index 3 then minIndex = 3, max = 25 at index 2 then maxIndex = 2

Swap 22 with element at index left. i.e., index 2

Swap 25 with element at index right. i.e., index 3 (already at index 3 after swapping 22)

Array after Pass 3: 11 12 **22** **25** 33 64

Now left is **3** and right **2**. and the condition $\text{left} < \text{right}$ failed and sorting complete.

Final Output

Original array:

64 25 12 22 11 33

Sorted array:

11 12 22 25 33 64

Rubric

Criteria	Marks
Dynamic memory allocation	1
Correct logic for finding both min and max in each pass	3
Correct implementation of the algorithm in C	4
Output printed in sample output format	2
Total	10

If above method not used, then total marks will be 0/10

Note:

- Your output should be printed as **sample output**
- **Reference:** <https://en.cppreference.com/w/c.html>
- Use of AI is strictly prohibited.